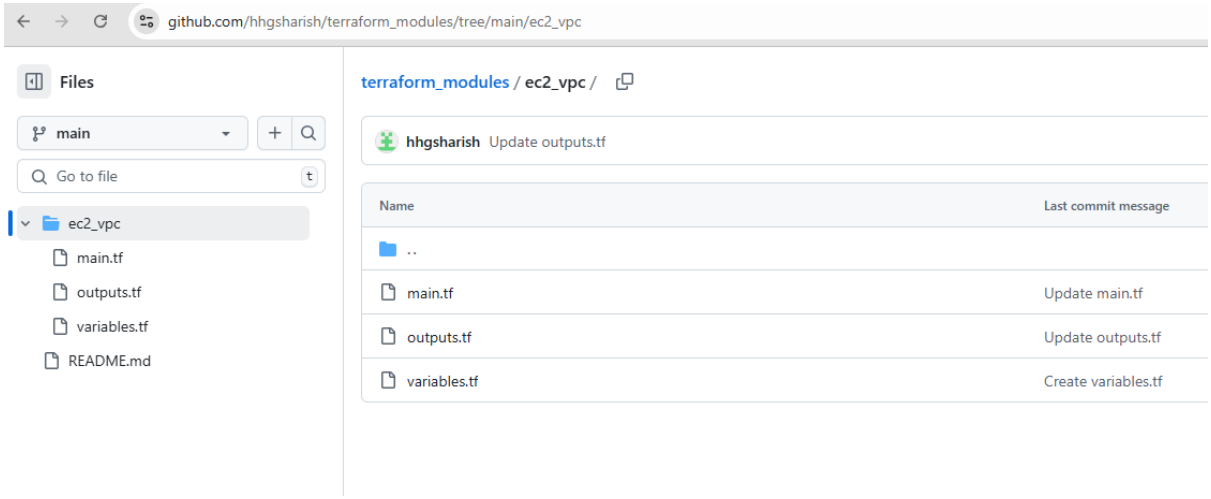
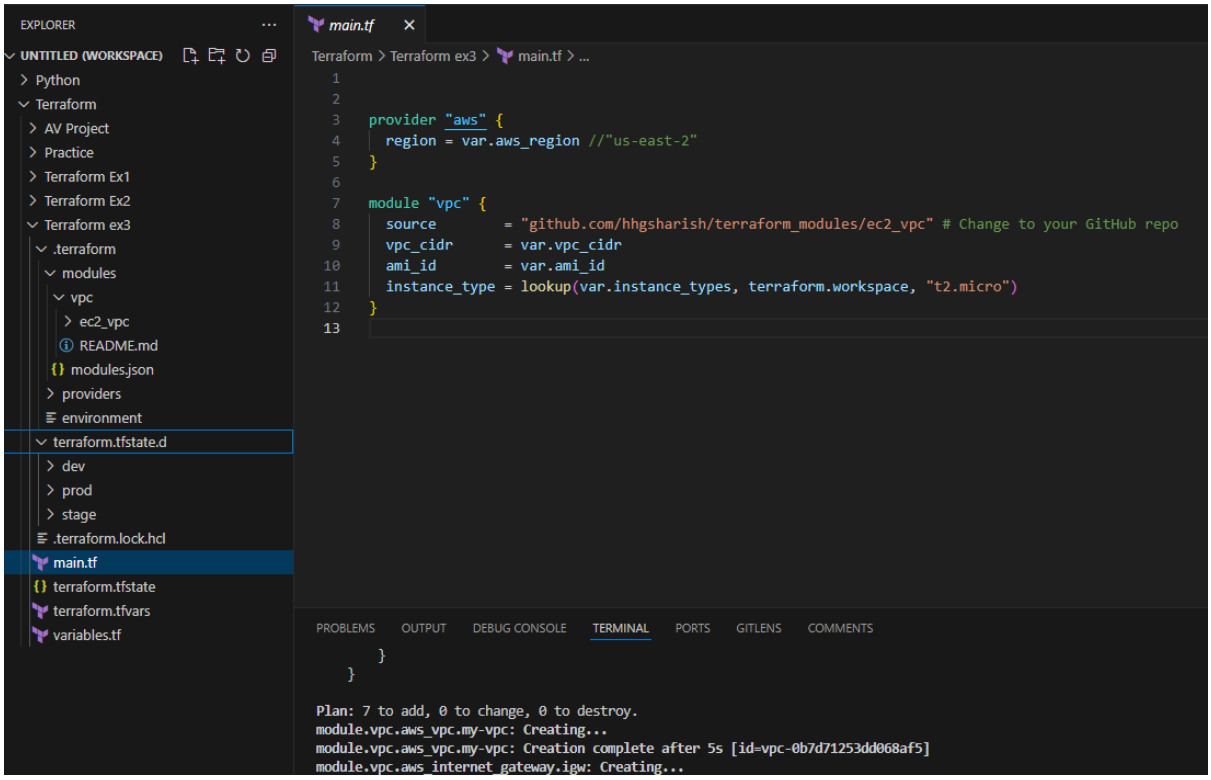
Modules & workspace assignment

- Create a Terraform module that provisions a VPC, security groups (allowing SSH & HTTP) , and an EC2 instance . Upload the module code to your github repo and reference the github source for your module block
- The setup should use Terraform workspaces for different environments (dev, stage, prod) and allow users to specify the CIDR block of their choice.

- Make use of -
- Variables for user-defined inputs.
- Locals for computed values.
- Functions to format values dynamically.





terraform.tfvars

Terraform > Terraform ex3 > terraform.tfvars > ami_id

```
aws_region = "us-east-2"
vpc_cidr   = "10.0.0.0/16"
ami_id     = "ami-0cb91c7de36eed2cb"
```

variables.tf

Terraform > Terraform ex3 > variables.tf > variable "ami_id" > type

```
variable "aws_region" {
  description = "AWS region"
  type        = string
}

variable "vpc_cidr" {
  description = "CIDR block for the VPC"
  type        = string
}

variable "ami_id" {
  description = "AMI ID for EC2"
  type        = string
}

variable "instance_types" {
  description = "Instance type based on the workspace"
  type        = map(string)

  default = {
    dev    = "t2.micro"
    stage  = "t3.small"
    prod   = "t3.medium"
  }
}
```

```
PS C:\devops\Terraform\Terraform ex3> terraform workspace new dev
Created and switched to workspace "dev"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\devops\Terraform\Terraform ex3> terraform init
Initializing the backend...
Initializing modules...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.89.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\devops\Terraform\Terraform ex3> terraform apply -auto-approve

Terraform used the selected providers to generate the following execution plan. Resource
  + create
```

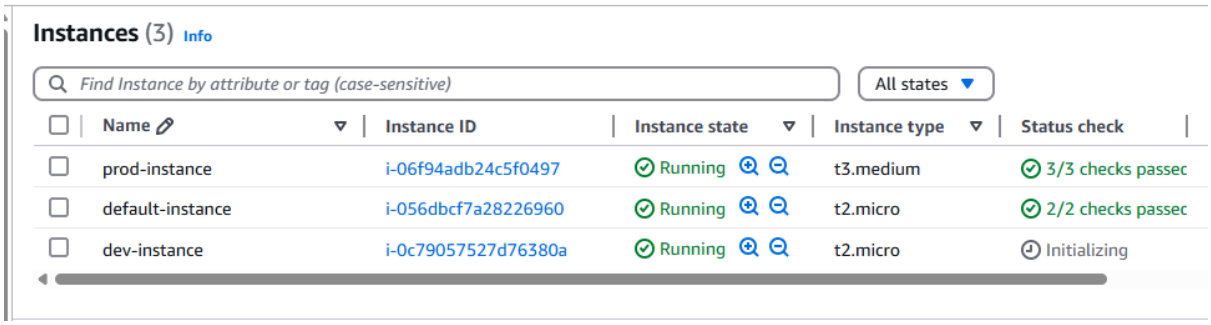
```
for this configuration.
PS C:\devops\Terraform\Terraform ex3> terraform workspace select prod
PS C:\devops\Terraform\Terraform ex3> terraform workspace show
prod
PS C:\devops\Terraform\Terraform ex3> terraform init
Initializing the backend...
Initializing modules...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.89.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\devops\Terraform\Terraform ex3> terraform apply -auto-approve

Terraform used the selected providers to generate the following execution plan. Resource
  + create
```


Instances (3) Info
Find Instance by attribute or tag (case-sensitive)
All states
Name
Instance ID
Instance state
Instance type
Status check
prod-instance
i-06f94adb24c5f0497
Running
t3.medium
3/3 checks passec
default-instance
i-056dbcf7a28226960
Running
t2.micro
2/2 checks passec
dev-instance
i-0c79057527d76380a
Running
t2.micro
Initializing